

UVa12387/LA5819 Alphabet Soup

题目链接

本题是[2011年ICPC欧洲区域赛西南赛区的A题](#)

题意

给出 S ($2 \leq S \leq 1000$) 个不同形状的符号，以及一个圆上的 P 个位置（用极角给出，等分成了360000个角度），选择 P 个不同的符号放在这 P 个位置上计算总共有多少种放置方案。注意一种方案如果可以旋转成另外一种方案，则视为同一种方案。输出方案数模100000007。

分析

本题虽说是第一道题目，但却是比较难的置换等价类计数题目。

注意，题目并未说 P 个位置已经排序好了，所以需要将 P 个极角位置排序。找出所有旋转后会重合的（旋转角度）方案，以统计答案。

对角度进行差分后，[本题目可以转化成字符串匹配问题，套用KMP模版可求解。](#)

下面将给出纯数论分析的解法。

首先容易想到一个结论：如果旋转角度为 x 时可以重合，那么旋转角度为 $k \cdot x$ (k 为正整数 $k \cdot x < 360000$) 时也能重合。

如果旋转角度为 x 时可以重合，记 $g = \gcd(360000, x)$, $x' = x \div g$, $m = 360000 \div g$, 则 $x' \perp m$ (x' 与 m 互质) 且每个置换分解循环的长度均为 m 。

记 $i = (x')^{-1} \bmod m$, 则在模 360000 剩余系下, $x \cdot 0i, x \cdot 1i, x \cdot 2i, x \cdot 3i, \dots, x \cdot (m-1)i$ 恰好为 $0, g, 2g, 3g, \dots, (m-1)g$ 。

可见每个置换分解循环对应的那些极角必然构成公差为 g 的等差数列。

因此得出另一个结论：若旋转角度为 x 时可以重合，则旋转角度为 $g = \gcd(360000, x)$ 时也可以重合。

两结论相结合，可以得出数论解法：枚举360000的真因子 a （共104个），判断以 a 为旋转角是否能重合，能重合则所有 a 的倍数 ($k \cdot a$) 均参与计数。

```

1  #include <iostream>
2  #include <algorithm>
3  using namespace std;
4
5  #define M 100000007
6  #define N 360000
7  #define T 104
8
9  int d[] = {1, 2, 3, 4, 5, 6, 8, 9, 10, 12, 15, 16, 18, 20, 24, 25, 30, 32, 36, 40, 50, 60, 64,
10      72, 75, 80, 90, 96, 100, 120, 125, 144, 150, 160, 180, 192, 200, 225, 240, 250, 288, 300,
11      320, 360, 375, 400, 450, 480, 500, 576, 600, 625, 720, 750, 800, 900, 960, 1000, 1125, 1200,
12      1250, 1440, 1500, 1600, 1800, 1875, 2000, 2250, 2400, 2500, 2880, 3000, 3600, 3750, 4000,
13      4500, 4800, 5000, 5625, 6000, 7200, 7500, 8000, 9000, 10000, 11250, 12000, 14400, 15000,
14      18000, 20000, 22500, 24000, 30000, 36000, 40000, 45000, 60000, 72000, 90000, 120000,
15      180000};
16
17 int a[N], inv[N+1], s, n, cc; long long pow[N+1], ans; bool vis[N];
18
19 int gcd(int a, int b, int& x, int& y) {
20     if (!b) {
21         x = 1; y = 0; return a;
22     } else {
23         int g = gcd(b, a%b, y, x);
24         y -= a/b*x;
25         return g;
26     }
27 }
28
29 int gcd(int a, int b) {
30     return b==0 ? a : gcd(b, a%b);
31 }
32
33 void rot(int s, int d) {
34     for (int i=1; i<n; ++i) if ((a[(i+s) % n] - a[i] + N) % N != d) return;
35     int g = gcd(s, n), p = n/g;
36     for (int i=s; i<n; i+=s) vis[i] = true, ans += pow[g*gcd(p, i/s)], ++cc;
37 }
38
39 long long solve() {
40     for (int i=0; i<n; ++i) cin >> a[i], vis[i] = false;
41     for (int i=1; i<=n; ++i) pow[i] = pow[i-1]*s % M;
42     sort(a, a+n);

```

```

41     for (int i=1; i<n; ++i) a[i] -= a[0];
42     a[0] = 0;
43     ans = pow[n]; cc = 1;
44     for (int i=0; i<T; ++i) {
45         int j = lower_bound(a, a+n, d[i]) - a;
46         if (j<n && a[j]==d[i] && !vis[j]) rot(j, d[i]);
47     }
48     return ans % M * inv[cc] % M;
49 }
50
51 int main() {
52     pow[0] = 1;
53     for (int i=1, x, y; i<=N; ++i) {
54         gcd(i, M, x, y);
55         inv[i] = (x+M) % M;
56     }
57     ios::sync_with_stdio(false);
58     while (cin >> s >> n && s >= 0) cout << solve() << endl;
59     return 0;
60 }

```